GameLang Uma linguagem específica de domínio com interpretação e geração de Python

Victor Will dos Santos - 194427
João Pedro Pirani - 194411
Ryan Markus Maciel Araujo - 202006
Lucas dos Santos Garreto - 200131
Enzo Giuliano Cardoso Santana - 200265

Projeto e Implementação de Compiladores
UNASP-SP
2026/2

Resumo

Este trabalho apresenta a GameLang, uma linguagem específica de domínio para regras básicas de jogos. O projeto implementa um compilador completo em Python, composto por análise léxica, parsing descendente, árvore sintática abstrata, análise semântica, representação intermediária serializável, otimizações, interpretador e geração de Python. A avaliação verifica diagnósticos com linha e coluna, equivalência entre duas rotas de execução e preservação de comportamento após otimização. A linguagem mantém um escopo deliberadamente pequeno, centrado em entidades, propriedades numéricas, colisões, dano, cura, movimento, repetição e saída textual. Os resultados da suíte automatizada mostram que o pipeline aceita os exemplos válidos, rejeita entradas adversariais e produz o mesmo estado observável no interpretador e no código gerado.

Palavras-chave: compiladores; linguagem específica de domínio; representação intermediária; geração de código; testes de equivalência.

1 Introdução

Regras simples de jogabilidade costumam exigir infraestrutura que desvia a atenção do comportamento descrito. A GameLang reduz esse custo para estudantes e criadores independentes ao representar entidades e ações com termos do domínio. A Entrega 1 definiu entidades globais, propriedades numéricas, colisões, dano, cura, movimento, repetição, saída textual e tradução para Python. O cronograma da disciplina acrescentou requisitos de análise, IR, runtime, otimização, reprodutibilidade e comunicação científica.

O objetivo deste trabalho foi construir uma implementação reproduzível que preservasse o escopo original e permitisse observar cada etapa do compilador. A contribuição prática é um repositório sem dependências externas de runtime, acompanhado de exemplos, casos inválidos, testes e uma interface de linha de comando.

2 Linguagem e método

A gramática usa declarações `entity`, eventos `when A touches B` e instruções para dano, cura, movimento, repetição e impressão. Expressões possuem precedência convencional e incluem valores inteiros, reais, booleanos, textos, posições e referências qualificadas.

O lexer percorre a fonte uma vez e anexa linha e coluna aos tokens. O parser descendente cria uma AST imutável. A análise semântica realiza duas etapas: registra entidades e propriedades; depois valida eventos, ações e expressões. Essa ordem impede que o backend receba referências inexistentes ou tipos incompatíveis.

A conversão para IR produz entidades inicializadas, instruções estruturadas e manipuladores de eventos. O formato JSON contém versão e posições de origem. O interpretador executa a IR, enquanto o backend emite Python que usa o mesmo runtime. Essa separação permite comparar resultados sem duplicar as regras de estado.

3 Otimizações

O otimizador implementa dobramento de constantes, identidades algébricas e eliminação de operações sem efeito. Por exemplo, `2 * (3 + 1)` se torna `8`, `x + 0` se torna `x`, e ações com quantidade zero são removidas. Um bloco `repeat 1` é substituído pelo próprio corpo.

Cada transformação opera sobre uma IR validada e registra sua categoria. A validação compara o snapshot de entidades e a saída textual antes e depois da otimização. O projeto evita dobrar divisões por zero, preservando o momento em que um erro de execução apareceria.

4 Avaliação

A suíte automatizada cobre lexer, comentários, textos, precedência, erros sintáticos, tabela de símbolos, tipos, serialização, colisões, interpretação, Python gerado e otimizações. Os testes de equivalência executam a mesma entrada pelo interpretador e por uma função Python gerada, então comparam propriedades e saída.

Os programas válidos incluem colisão, movimento repetido, dano seguido de cura e um caso direcionado às otimizações. Os exemplos inválidos provocam caractere desconhecido, ponto e vírgula ausente, entidade inexistente, propriedade inexistente e tipo incompatível. Todos os diagnósticos originados no frontend carregam linha e coluna.

5 Discussão

Verificação	Resultado	Evidência
Suíte automatizada	27 aprovados	Execução com unittest
Programas válidos	4	Exemplos na pasta examples
Programas inválidos	5	Erros léxicos, sintáticos e semânticos
Rotas equivalentes	Aprovado	Snapshot do interpretador e do Python
Classes de otimização	3	Constantes, identidades e código sem efeito

A IR estruturada foi suficiente porque o núcleo não possui condicionais nem saltos arbitrários. Um CFG ou SSA adicionaria complexidade sem melhorar as transformações atuais. O desenho mantém espaço para uma extensão futura com condicionais, variáveis locais e um CFG explícito.

O uso de um runtime comum reduz divergências entre interpretação e geração. A dependência também significa que o arquivo Python gerado não é totalmente independente. A distribuição pode instalar o pacote ou empacotar o runtime junto ao programa.

6 Limitações

A linguagem não executa gráficos ou física, não detecta colisões geométricas e não possui funções ou módulos. Propriedades iniciais são constantes numéricas. O evento de toque deve ser informado ao runtime. As medições do projeto demonstram corretude nos casos cobertos, mas não sustentam conclusões amplas sobre desempenho.

7 Conclusão

A implementação atende ao fluxo técnico previsto: fonte, lexer, parser, AST, análise semântica, IR, otimização e duas rotas de execução. A combinação de posições de origem, formato intermediário legível e testes de equivalência torna o projeto adequado ao ensino de compiladores e à demonstração acadêmica. Trabalhos futuros podem acrescentar condicionais, CFG, SSA e uma distribuição autônoma do código gerado.

Referências

AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Addison-Wesley, 2006.

FOWLER, M. Domain-specific languages. Boston: Addison-Wesley, 2010.

GAME LANG TEAM. *GameLang: especificação da linguagem, Entrega 1*. Documento interno da disciplina, 2026.

UNASP-SP. *Cronograma do Projeto GameLang: Projeto e Implementação de Compiladores 2026/2*. Documento da disciplina, 2026.